

PROJECT DOCUMENTATION

FLOOD EVACUATION PATH PLANNING SYSTEM V2.0

Sustainable Development Goal (SDG 11: Sustainable Cities)

Submitted to:

Professor Justin Neypes

Course:

IT Elective 1

Academic Term:

2nd Semester/ CY 2/ 2025-2026

Prepared by:

Flood Buster

Members:

Caliguiran, Arjane

Esteban, Maria Victoria

Lagrimas, Don Christian

Lamayo, Justine Kurt

Riza, Christina Alexandra

May 22, 2026



I. INTRODUCTION

A. PROJECT OVERVIEW

FloodBuster is a high-quality VB.NET Windows Forms Application created as a component of the ITELEC1 capstone project. The system aims to aid disaster readiness and response by equipping local government units, barangay officials, and community members with a unified resource for monitoring floods, managing emergency alerts, and planning evacuation routes.

The application handles essential flood-related information using a persistent SQL Server database, provides decision-support insights through a reporting module, and implements role-based access to safeguard sensitive operational data. It is designed with a complete N-Tier separation pattern (Presentation, Business Logic, and Data Access layers).

B. SELECTED UN SDG OBJECTIVE

FloodBuster directly tackles UN Sustainable Development Goal 11: Sustainable Cities and Communities, focusing on Target 11.5:

"By 2030, greatly lower the death toll and the number of individuals impacted, while markedly reducing direct economic losses as a percentage of global GDP due to disasters, particularly water-related ones, emphasizing the



protection of the impoverished and those in vulnerable circumstances."

Floods rank as one of the most damaging and common natural disasters in the Philippines, notably impacting low-income urban barangays. FloodBuster offers a data-centric platform that facilitates quicker, safer, and more systematic evacuation choices, directly supporting this objective.

II. REQUIREMENTS ANALYSIS

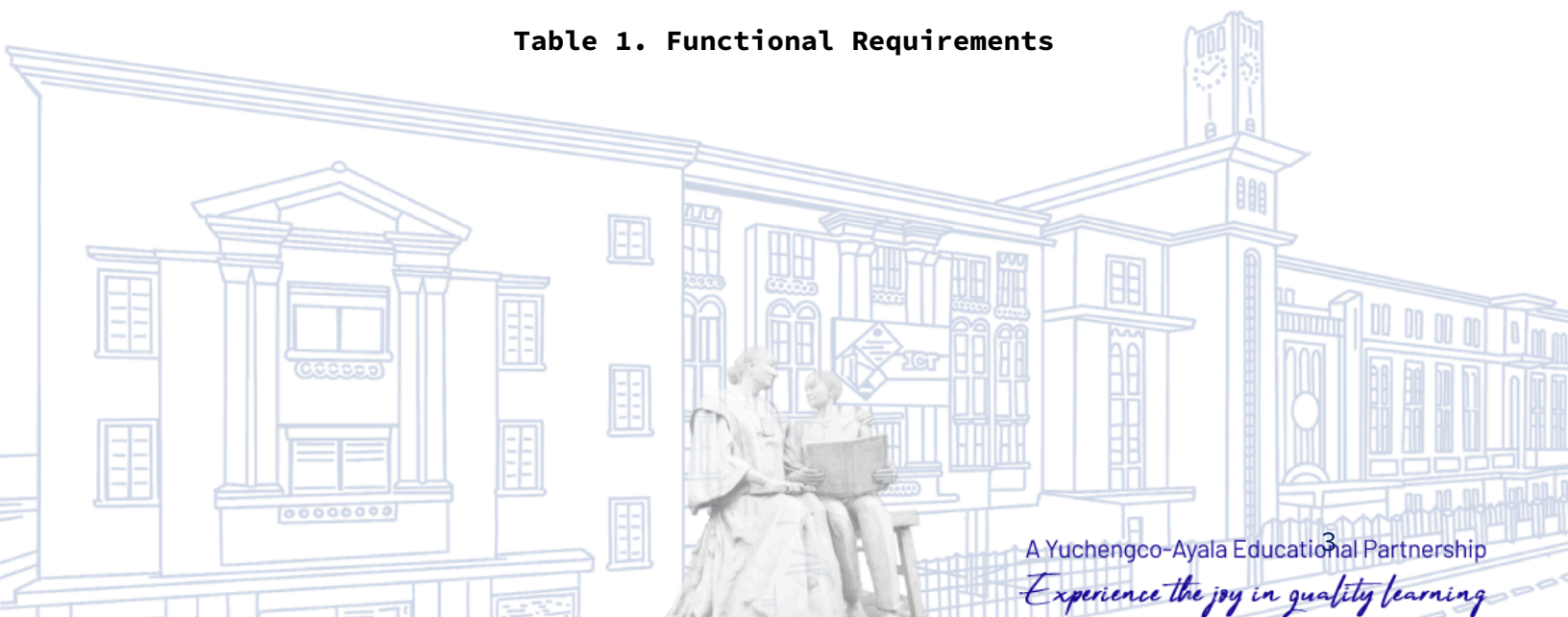
2.1 FUNCTIONAL REQUIREMENTS(FR)

ID	Requirements	Alignment to Code
FR1	(Data Persistence): The system must perform full CRUD operations (Create, Read, Update, Delete) on a SQL Server database using parameterized queries (to prevent SQL injection).	The system performs full CRUD operations on a SQL Server database using SqlCommand.Parameters.AddWithValue (e.g., in Add, MarkFlooded, Update and Delete functions) to effectively prevent SQL injection.
FR2	(Business Logic): The system must execute specific logic (e.g., calculating carbon offsets, scholarship eligibility, or health	The system architecture adheres to a modular design by segregating operations into dedicated Repository Classes (e.g., UserRepository.vb , BarangayRepository.vb , and AlertRepository.vb).



	risks) within dedicated Class modules.	
FR3	(Reporting): The system must generate a professional report summarizing data trends (e.g., Monthly Resource Consumption Report).	The system integrates Crystal Reports (FloodReport.rpt) to facilitate the generation of structured data summaries. The ReportForm.vb utilizes DateTimePicker controls to enable precise temporal filtering for the "Monthly Flood Incident Summary".
FR4	(Security): Implementation of a Login System with basic user role awareness (Admin vs. Standard User).	Enforced via the ValidateLogin function in UserRepository.vb , which retrieves the Role attribute from the database. The system dynamically manages the .Visible property of administrative controls based on whether the authenticated session is identified as Admin or StandardUser.

Table 1. Functional Requirements





2.2 NON-FUNCTIONAL REQUIREMENTS (NFR)

ID	Requirements	Alignment to Code
NFR1	(UI Consistency): UI must follow standard naming conventions (e.g., btnSubmit, lblStatus) and maintain a clean, accessible layout.	The frontend documentation specifies a standardized naming convention (e.g., btnSubmit, dgvBarangays) to ensure an accessible and maintainable interface.
NFR2	(Maintainability): Code must be documented with XML comments or standard green-line comments explaining complex logic.	The implementation incorporates XML documentation tags (e.g., '''<summary>') and standard comments within the class modules to describe the underlying logic and enhance code readability.
NFR3	(Reliability): Database connection strings must be managed via configuration files or a dedicated Connection Class.	A centralized DatabaseConnection.vb class is employed to manage SQL connection strings. This prevents hardcoding across multiple forms and ensures a single source of truth for database connectivity.

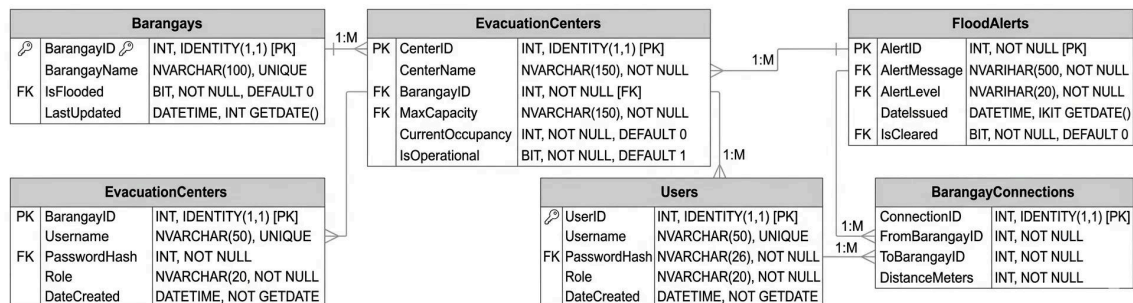
Table 2. Non-Functional Requirements



III. DATABASE DESIGN

3.1. ENTITY RELATIONSHIP DIAGRAM (ERD)

The following diagram describes the logical relationships between the core database entities.



Picture 1. Physical ERDs of the FloodBuster 2.0

1. Barangays and EvacuationCenters (1:M)

- Relationship: One Barangay can host multiple EvacuationCenters.
- Physical Link: The BarangayID in the EvacuationCenters table is a Foreign Key (FK) that references the BarangayID in the Barangays table.
- Constraint: CONSTRAINT FK_Evacuation_Barangay FOREIGN KEY (BarangayID) REFERENCES Barangays(BarangayID).

2. Barangays and FloodAlerts (1:M)

- Relationship: One Barangay can have many FloodAlerts issued over time.



- Physical Link: The BarangayID in the FloodAlerts table is a FK pointing back to the parent Barangays table.
- Constraint: CONSTRAINT FK_Alerts_Barangay FOREIGN KEY (BarangayID) REFERENCES Barangays(BarangayID).

3. BarangayConnections (Self-Referencing M:N)

This is the most complex part of our design. It uses a "Link" or "Associative" table to connect a Barangay to other Barangays (to map a network of paths or distances).

- Relationship: A Barangay can be connected to many other Barangays.
- Physical Link: The BarangayConnections table has two Foreign Keys:
 - FromBarangayID → references BarangayID.
 - ToBarangayID → references BarangayID.
- Constraint: FK_From_Barangay and FK_To_Barangay.

4. Users (Independent)

- Relationship: the Users table is unconnected (orphaned) from the rest of the tables.

3.2. DATA DICTIONARY

Table: Barangays

Column Name	Data Type	Constraints	Description
BarangayID	INT	PRIMARY KEY, IDENTITY(1,1)	Unique identifier for each barangay node.



Column Name	Data Type	Constraints	Description
BarangayName	NVARCHAR(100)	NOT NULL, UNIQUE	Name of the barangay.
IsFlooded	BIT	NOT NULL, DEFAULT 0	Flood status flag: 1 = Flooded, 0 = Safe.
LastUpdated	DATETIME	NOT NULL, DEFAULT GETDATE()	Timestamp of the last flood status update.

Table: EvacuationCenters

Column Name	Data Type	Constraints	Description
CenterID	INT	PRIMARY KEY, IDENTITY(1,1)	Unique identifier for each evacuation center.
CenterName	NVARCHAR(150)	NOT NULL	Name of the evacuation center.
BarangayID	INT	FOREIGN KEY (Barangays)	The barangay where the center is located.
MaxCapacity	INT	NOT NULL	Maximum number of evacuees the center can hold.
CurrentOccupancy	INT	NOT NULL, DEFAULT 0	Current number of registered evacuees.
IsOperational	BIT	NOT NULL, DEFAULT 1	Operational status of the center.

Table: FloodAlerts



IT ELECTIVE 1
School of Arts, Science & Technology
National Teachers College

Column Name	Data Type	Constraints	Description
AlertID	INT	PRIMARY KEY, IDENTITY(1,1)	Unique identifier for each alert.
BarangayID	INT	FOREIGN KEY (Barangays)	The barangay the alert is issued for.
AlertMessage	NVARCHAR(500)	NOT NULL	The text content of the emergency alert.
AlertLevel	NVARCHAR(20)	NOT NULL	Severity level: LOW, MODERATE, CRITICAL.
DateIssued	DATETIME	NOT NULL, DEFAULT GETDATE()	Timestamp when the alert was issued.
IsCleared	BIT	NOT NULL, DEFAULT 0	Whether the alert has been cleared.

Table: Users

Column Name	Data Type	Constraints	Description
UserID	INT	PRIMARY KEY, IDENTITY(1,1)	Unique identifier for each user.
Username	NVARCHAR(50)	NOT NULL, UNIQUE	Login username.
PasswordHash	NVARCHAR(256)	NOT NULL	Hashed password for secure authentication.
Role	NVARCHAR(20)	NOT NULL	User role: Admin or StandardUser.



IT ELECTIVE 1
School of Arts, Science & Technology
National Teachers College

Column Name	Data Type	Constraints	Description
DateCreated	DATETIME	NOT NULL, DEFAULT GETDATE()	Account creation timestamp.

Table: BarangayConnections (Road Network)

Column Name	Data Type	Constraints	Description
ConnectionID	INT	PRIMARY KEY, IDENTITY(1,1)	Unique identifier for the road connection.
FromBarangayID	INT	FOREIGN KEY (Barangays)	Origin barangay node.
ToBarangayID	INT	FOREIGN KEY (Barangays)	Destination barangay node.
DistanceMeters	INT	NOT NULL	Road distance in meters between the two barangays.

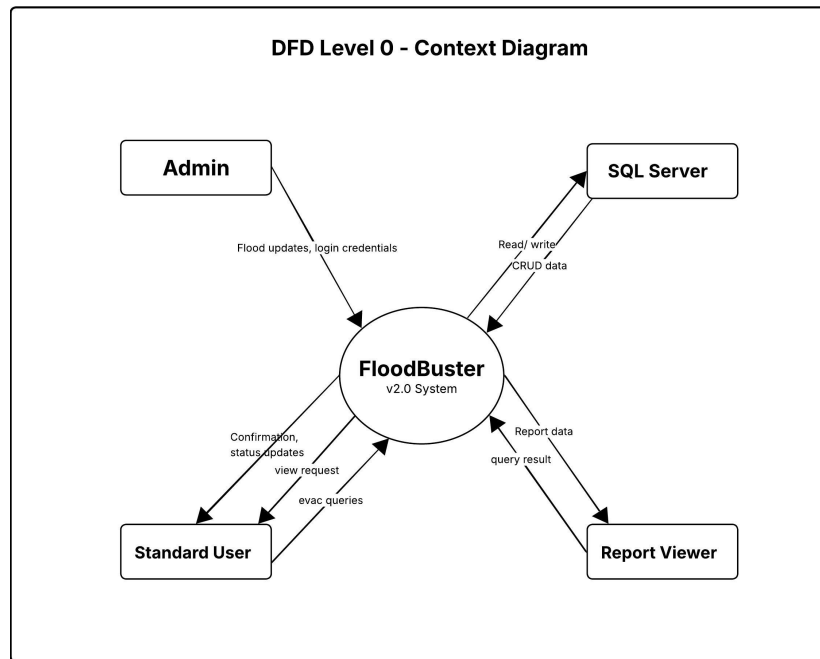


A Yuchengco-Ayala Educational Partnership
Experience the joy in quality learning



IV. SYSTEM ARCHITECTURE

4.1. DFD LEVEL 0 – CONTEXT DIAGRAM

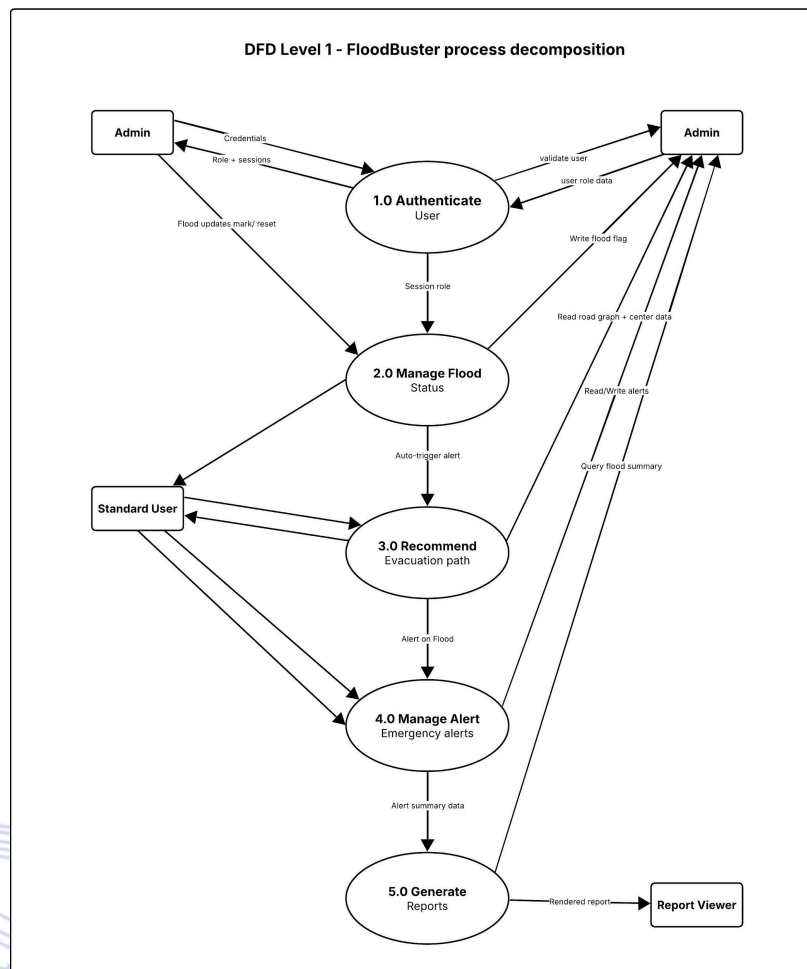


The Level 0 Data Flow Diagram acts as a high-level bird's-eye view that treats the entire FloodBuster system as a single central process, focusing entirely on how it interacts with external boundaries rather than its internal mechanics. In this diagram, the system exchanges data with four distinct external entities: the Admin (Barangay Official), who provides credentials and flood updates to receive confirmations and updated displays; the Standard User (Resident), who submits credentials and evacuation queries to receive status info and recommendations; the SQL Server Database, which accepts data writes and returns requested records; and the Crystal Reports engine, which receives



raw query inputs to return rendered PDF summaries. By completely omitting internal subprocesses, this context diagram strictly maps out what inputs the system requires and what outputs it produces.

4.2. DFD LEVEL 1 – FLOODBUSTER PROCESS DECOMPOSITION



The Level 1 Data Flow Diagram opens up the central Level 0 system circle to map out the explicit data pathways and internal logic across five distinct, interconnected processes. It begins with



Process 1.0 (Authenticate User), which validates incoming credentials against the SQL Server Users table to establish the session role that authorizes all subsequent actions. From there, the data splits based on privileges: **Process 2.0 (Manage Flood Status)** allows Admins to write flood flags to the database, which automatically triggers **Process 4.0 (Manage Emergency Alerts)** to generate a critical warning that both roles can view but only Admins can clear. Concurrently, **Process 3.0 (Recommend Evacuation Path)** pulls network connections to execute Dijkstra's algorithm for residents, while **Process 5.0 (Generate Report)** compiles alert histories and database records to feed Crystal Reports, detailing exactly how data transforms as it shifts between users, processes, and storage tables.

V. MODULE BREAKDOWN – DEFINE EACH FORM.VB CLASS AND HOW THEY INTERACT

PRESENTATION LAYER

a. DashboardForm.vb

```
Public Class DashboardForm
```

- This starts the DashboardForm class.
- The Dashboard is the main menu of the FloodBuster system after login.

```
Public isAdmin As Boolean
```

- This variable stores whether the logged-in user is an Admin or not.
 - **True** → Admin



- **False** → Standard User
- This controls which buttons/features are visible.

```
Private Sub DashboardForm_Load(sender As Object, e As EventArgs) Handles MyBase.Load
```

- This event automatically runs when the DashboardForm opens.
- It is used to:
 - Set labels
 - Configure buttons
 - Center controls
 - Set form position

```
If isAdmin Then  
    lblWelcome.Text = "Welcome to the Admin Dashboard!"  
    lblsubtitle.Text = "Admin Controls"  
Else  
    lblWelcome.Text = "Welcome to the Dashboard!"  
    lblsubtitle.Text = "User Options"
```

- This checks the user's role.
 - Algorithm:
 - If the user is an Admin:
 - Show Admin welcome message
 - Show "Admin Controls"
 - Otherwise:
 - Show normal dashboard message
 - Show "User Options"
- This is role-based UI logic.

```
ConfigureButtonVisibility()
```

- Calls the method that decides which buttons should be visible depending on the user role.

```
CenterEverything()
```

- Calls the method that centers all UI controls inside the form.



```
Me.StartPosition = FormStartPosition.CenterScreen
```

- Makes the DashboardForm appear at the center of the screen when opened.

```
Private Sub DashboardForm_Resize(sender As Object, e As EventArgs) Handles Me.Resize  
    CenterEverything()  
End Sub
```

- Whenever the window size changes, the controls are automatically centered again.
- This keeps the UI responsive and organized.

```
Private Sub ConfigureButtonVisibility()
```

- This method controls which buttons are shown based on the user role.

```
btnMarkFlooded.Visible = isAdmin  
btnManageAlerts.Visible = isAdmin  
btnGenerateReport.Visible = isAdmin  
btnResetFlood.Visible = isAdmin
```

- Admin-only buttons.
- If **isAdmin = True**:
 - Buttons become visible
- If **False**:
 - Buttons are hidden
- These features are restricted because they can modify system data.

```
btnViewFloodStatus.Visible = Not isAdmin  
btnEvacuationPath.Visible = Not isAdmin  
btnViewAlerts.Visible = Not isAdmin
```

- These buttons are shown only for standard users.
- **Not isAdmin** means:
 - If user is NOT admin → show buttons

```
Public Sub CenterEverything()
```



- This method aligns all UI elements properly in the center of the form.

```
flpButtons.AutoSize = True  
flpButtons.MaximumSize = New Size(700, 0)
```

- Makes the FlowLayoutPanel automatically adjust its size while limiting its width.

```
pn1.Left = (Me.ClientSize.Width - pn1.Width) / 2  
pn1.Top = (Me.ClientSize.Height - pn1.Height) / 2
```

- Centers the main panel (**pn1**) horizontally and vertically.
- Algorithm:
 - Get form width
 - Subtract panel width
 - Divide by 2
 - Position panel in center

```
lblWelcome.Left = (pn1.Width - lblWelcome.Width) / 2  
lblWelcome.Top = 30
```

- Centers the Welcome label inside the panel.

```
lblsubtitle.Left = (pn1.Width - lblsubtitle.Width) / 2  
lblsubtitle.Top = lblWelcome.Bottom + 5
```

- Places the subtitle below the Welcome label with spacing.

```
flpButtons.Left = (pn1.Width - flpButtons.Width) / 2  
flpButtons.Top = lblsubtitle.Bottom + 35
```

- Centers the button container below the subtitle.

```
btnLogout.Left = pn1.Width - btnLogout.Width - 20  
btnLogout.Top = pn1.Height - btnLogout.Height - 20
```



IT ELECTIVE 1
School of Arts, Science & Technology
National Teachers College

- Places the Logout button at the bottom-right corner of the panel.

```
Private Sub btnViewFloodStatus_Click(sender As Object, e As EventArgs) Handles btnViewFloodStatus.Click
    Dim frm As New BarangayForm
    frm.IsAdminMode = False
    frm.Show()
    Me.Hide()
```

- When clicked:
 - Opens BarangayForm
 - Hides DashboardForm
- This is form navigation logic.

```
Private Sub btnEvacuationPath_Click(sender As Object, e As EventArgs) Handles btnEvacuationPath.Click
    EvacuationForm.Show()
    Me.Hide()
```

- Opens the evacuation recommendation form.

```
Private Sub btnViewAlerts_Click(sender As Object, e As EventArgs) Handles btnViewAlerts.Click
```

- Opens AlertForm in User View mode.

```
AlertForm.IsAdminMode = False
```

- Hides the Clear Alerts button because standard users are not allowed to clear alerts.

```
AlertForm.Label.Text = "View Flood Alerts"
```

- Changes the title label for user mode.

```
frm.Show()
Me.Hide()
```

- Shows AlertForm and hides DashboardForm.

```
Private Sub btnMarkFlooded_Click(sender As Object, e As EventArgs) Handles btnMarkFlooded.Click
    Dim frm As New BarangayForm
    frm.IsAdminMode = True
    frm.Show()
    Me.Hide()
```




IT ELECTIVE 1
School of Arts, Science & Technology
National Teachers College

- Allows the Admin to open BarangayForm for flood management.

```
Private Sub btnManageAlerts_Click(sender As Object, e As EventArgs) Handles btnManageAlerts.Click
    ' IsAdminMode = True shows pnlStatus and btnClearAlerts inside AlertForm_Load
    AlertForm.IsAdminMode = True
    AlertForm.Label.Text = "Manage System Alerts"
    AlertForm.Show()
    Me.Hide()
```

- Opens AlertForm in Admin mode.

```
AlertForm.Label.Text = "Manage System Alerts"
```

- Changes the label title for admin mode.

```
Private Sub btnResetFlood_Click(sender As Object, e As EventArgs) Handles btnResetFlood.Click
```

- Runs when the Admin clicks the Reset Flood button.

```
Dim result = MessageBox.Show("Reset all flood statuses to normal?", "Confirm Reset",
    MessageBoxButtons.YesNo, MessageBoxIcon.Warning)
```

- Shows a confirmation dialog before resetting all flood statuses.
- This prevents accidental resets.

```
If result = DialogResult.Yes Then
```

- Only continue if the user clicks Yes.

```
MessageBox.Show("All statuses have been reset.", "Success",
    MessageBoxButtons.OK, MessageBoxIcon.Information)
```

- Displays a success message after the database update is completed.

```
Private Sub btnLogout_Click(sender As Object, e As EventArgs) Handles btnLogout.Click
    LoginForm.Show()
    Me.Close()
```

- Logs the user out.
- Algorithm:



- Return to LoginForm
- Close DashboardForm

```
Private Sub pnl1_Paint(sender As Object, e As PaintEventArgs) Handles pnl1.Paint  
End Sub
```

- Automatically generated Paint event.
- Currently empty and has no function.

b. LoginForm.vb

```
Public Class LoginForm
```

- Starts the LoginForm class.
- This form handles user authentication.

```
Private Sub LoginForm_Load(sender As Object, e As EventArgs) Handles MyBase.Load  
CenterPanel()
```

- Runs automatically when the LoginForm opens.
- Calls the **CenterPanel()** method to center the login panel on the screen.

```
Private Sub LoginForm_Resize(sender As Object, e As EventArgs) Handles Me.Resize  
CenterPanel()
```

- Runs whenever the form size changes.
- Keeps the login panel centered even if the window is resized.

```
Private Sub CenterPanel()  
Panel1.Left = (Me.ClientSize.Width - Panel1.Width) / 2  
Panel1.Top = (Me.ClientSize.Height - Panel1.Height) / 2  
End Sub
```

- Centers **Panel1** horizontally and vertically.
- Algorithm:
 - Get the width and height of the form
 - Subtract the panel size



- Divide by 2
- Position the panel in the center
- This improves the UI layout.

```
Private Sub btnLogin_Click(sender As Object, e As EventArgs) Handles btnLogin.Click
```

- Runs when the Login button is clicked.
- This is the main login process of the system.

```
Dim userSvc As New UserService()
```

- Creates an instance of the **UserService** class.
- **UserService** contains the business logic for:
 - Login
 - Validation
 - Authentication

```
Dim role As String = userSvc.Login(txtUsername.Text, txtPassword.Text)
```

- Sends the entered username and password to the **Login()** method inside **UserService**.
 - Algorithm:
 - Read username from **txtUsername**
 - Read password from **txtPassword**
 - Send values to **UserService**
 - Receive the user's role
 - Possible returned values:
 - "Admin"
 - "StandardUser"
 - "" (failed login)

```
If role <> "" Then
```

- Checks if login was successful.
- If the returned role is not empty:
 - User exists
 - Login succeeds

```
Dim isAdmin As Boolean = (role.Equals("Admin", StringComparison.OrdinalIgnoreCase))
```



- Checks if the returned role is **"Admin"**.
- **OrdinalIgnoreCase** ignores uppercase/lowercase differences.

```
DashboardForm.isAdmin = isAdmin
```

- Passes the Admin status to **DashboardForm**.
- This allows the Dashboard to:
 - Show Admin controls
 - Hide restricted features for standard users

```
MessageBox.Show("Login Successful! Welcome, " & txtUsername.Text, "FloodBuster",  
    MessageBoxButtons.OK, MessageBoxIcon.Information)
```

- Displays a success message after successful login.
- The message includes the entered username.

```
DashboardForm.Show()  
Me.Hide()
```

- Opens the DashboardForm and hides the LoginForm.
- This moves the user into the main system.

Else

- Runs if login fails.
- This means:
 - Username/password is incorrect
 - No matching account exists

```
MessageBox.Show("Invalid username or password recorded in our database.", "Login Error",  
    MessageBoxButtons.OK, MessageBoxIcon.Error)
```

- Displays an error message for invalid login credentials.

```
txtPassword.Clear()
```

- Clears the password textbox after failed login.



- Improves security and usability.

```
txtPassword.Focus()
```

- Places the cursor back inside the password textbox.
- Allows the user to quickly try logging in again.

```
Private Sub Label1_Click(sender As Object, e As EventArgs) Handles Label1.Click  
:  
End Sub
```

- Automatically generated event for clicking **Label1**.
- Currently empty and has no function.

```
Private Sub Button1_Click(sender As Object, e As EventArgs) Handles logo.Click  
:  
End Sub
```

- Automatically generated click event for the **logo** button or image.
- Currently empty and has no functionality.

c. BarangayForm.vb

```
Public Class BarangayForm
```

- Starts the BarangayForm class.
- This form manages:
 - Viewing barangays
 - Flood monitoring
 - Searching/filtering
 - Admin flood actions

```
Public Property IsAdminMode As Boolean = False
```

- Stores whether the form is running in Admin mode.
- **True** → Admin features enabled



- **False** → Public/User view only

```
Private _svc As New FloodStatusService()
```

- Creates an instance of **FloodStatusService**.
- This service handles the business logic for:
 - Loading barangays
 - Marking flooded areas
 - Adding barangays
 - Deleting barangays

```
Private barangayData As DataTable
```

- Stores barangay data in memory.
- This allows:
 - Faster searching
 - Faster filtering
 - Less database calls

```
Private showOnlyFlooded As Boolean = False
```

- Tracks whether the flood filter is enabled.
 - **True** → Show flooded only
 - **False** → Show all barangays

```
Me.StartPosition = FormStartPosition.CenterScreen
```

- Makes the form open at the center of the screen.

```
ApplyViewSettings()
```

- Applies UI settings depending on Admin/User mode.
- Controls:
 - Button visibility
 - Labels
 - Read-only settings

```
RefreshBarangayList()
```




- Loads and refreshes the barangay list from the service layer.

```
barangayData = _svc.GetAll()
```

- Retrieves all barangay records using **FloodStatusService**.
- The returned data is stored inside **barangayData**.

```
dgvBarangays.DataSource = barangayData
```

- Displays the barangay records inside the DataGridView.

```
If dgvBarangays.Columns.Contains("BarangayName") Then
```

- Checks if the DataGridView contains the expected columns before formatting them.
- Prevents errors.

```
dgvBarangays.Columns("BarangayID").HeaderText = "ID"
```

- Changes the column header text from **BarangayID** to **ID**.

```
dgvBarangays.Columns("BarangayID").Width = 50
```

- Sets the width of the ID column to 50 pixels.

```
dgvBarangays.Columns("BarangayName").HeaderText = "Barangay Name"
```

- Changes the column title to a more readable label.

```
dgvBarangays.Columns("BarangayName").AutoSizeMode = DataGridViewAutoSizeColumnMode.Fill
```

- Makes the Barangay Name column automatically fill available space.



```
dgvBarangays.Columns("IsFlooded").HeaderText = "Flooded?"
```

- Changes the column title for flood status.

```
dgvBarangays.Columns("LastUpdated").HeaderText = "Last Updated"
```

- Changes the column title for the update timestamp.

```
Private Sub txtSearch_TextChanged(sender As Object, e As EventArgs) Handles txtSearch.TextChanged  
    PerformSearchAndFilter()  
End Sub
```

- Runs automatically whenever the user types in the search textbox.
- Calls the filtering/search logic in real time.

```
showOnlyFlooded = Not showOnlyFlooded
```

- Toggles the filter state.
- Algorithm:
 - If currently False → becomes True
 - If currently True → becomes False

```
If showOnlyFlooded Then
```

- Checks if flood-only filtering is enabled.

```
btnFilterFlooded.Text = "Showing: Flooded"
```

- Updates the button text to indicate the active filter.

```
btnFilterFlooded.BackColor = Color.LightCoral
```

- Changes the button color to light red to visually indicate flood filtering.

```
btnFilterFlooded.Text = "Show Only Flooded"
```



- Restores the original button text when the filter is disabled.

```
btnFilterFlooded.BackColor = Color.FromKnownColor(KnownColor.Control)
```

- Restores the default button color.

```
Private Sub PerformSearchAndFilter()
```

- Runs the combined search and filter logic after changing filter settings.

```
If barangayData IsNot Nothing Then
```

- Checks if barangay data exists before filtering.
- Prevents null reference errors.

```
Dim dv As New DataView(barangayData)
```

- Creates a **DataView** for filtering and searching data dynamically.

```
Dim filterString As String = String.Format("BarangayName LIKE '%{0}%'", txtSearch.Text.Replace("'", ""))
```

- Creates a search filter using SQL-style LIKE matching.
- Algorithm:
 - Read search text
 - Search inside BarangayName
 - Match partial names
- Example:
 - Typing **"San"** finds:
 - San Jose
 - San Roque
 - San Rafael

```
txtSearch.Text.Replace("'", "")
```



- Escapes apostrophes to prevent filter syntax errors.

```
If showOnlyFlooded Then  
    filterString &= " AND IsFlooded = 1"  
End If
```

- Adds an additional condition to show only flooded barangays.
- Algorithm:
 - Check if filter is enabled
 - Add flood condition to filter string

```
dv.RowFilter = filterString
```

- Applies the search and filter conditions to the DataView.

```
dgvBarangays.DataSource = dv
```

- Displays the filtered results inside the DataGridView.

```
Catch ex As Exception  
    MessageBox.Show("Filter Error: " & ex.Message)
```

- Handles filter/search errors.
- Displays the error message if filtering fails.

```
If _svc.AddBarangay(txtBarangayName.Text) Then
```

- Sends the entered barangay name to **FloodStatusService**.
- If successful:
 - Add barangay to database

```
txtBarangayName.Clear()
```

- Clears the textbox after successfully adding a barangay.



RefreshBarangayList()

- Reloads the DataGridView to show updated records.

```
...  
If dgvBarangays.CurrentRow IsNot Nothing Then
```

- Checks if the user selected a row in the DataGridView.

```
Dim id As Integer = Convert.ToInt32(dgvBarangays.CurrentRow.Cells("BarangayID").Value)
```

- Gets the selected barangay's ID.

```
Dim name As String = dgvBarangays.CurrentRow.Cells("BarangayName").Value.ToString()
```

- Gets the selected barangay's name.

```
If _svc.MarkFlooded(id, name) Then
```

- Calls the MarkFlooded() method inside FloodStatusService.
- Business Logic:
 - Update flood status
 - Automatically create a CRITICAL alert

```
MessageBox.Show(name & " marked as flooded.", "Success",  
    MessageBoxButtons.OK, MessageBoxIcon.Information)
```

- Displays confirmation after successfully marking a barangay as flooded.

```
If MessageBox.Show("Are you sure you want to delete this?", "Confirm",  
    MessageBoxButtons.YesNo) = DialogResult.Yes Then
```

- Shows a confirmation dialog before deleting a barangay.
- Prevents accidental deletion.

```
If _svc.DeleteBarangay(id) Then
```




- Deletes the selected barangay using **FloodStatusService**.

```
lblMode.Visible = IsAdminMode
```

- Shows the Admin mode label only for Admin users.

```
btnAdd.Visible = IsAdminMode  
btnMarkFlooded.Visible = IsAdminMode  
btnDelete.Visible = IsAdminMode
```

- Only Admins can:
 - Add barangays
 - Mark flooded areas
 - Delete barangays

```
lblmode2.Visible = Not IsAdminMode
```

- Shows the public view label only for standard users.

```
dgvBarangays.Visible = True  
btnBack.Visible = True
```

- Makes the DataGridView and Back button visible for all users.

```
If IsAdminMode Then
```

- Checks whether the form is running in Admin mode.

```
lblMode.Text = "Flood Management"
```

- Displays the Admin panel title.

```
Me.Text = "FloodBuster - Administrator"
```

- Changes the window title for Admin mode.



```
dgvBarangays.ReadOnly = False
```

- Allows Admins to edit/interact with the DataGridView.

```
lblmode2.Text = "Current Flood Status"
```

- Displays the public flood monitoring title.

```
Me.Text = "FloodBuster - Public View"
```

- Changes the form title for public users.

```
dgvBarangays.ReadOnly = True
```

- Prevents standard users from editing data.

```
DashboardForm.Show()  
Me.Close()
```

- Returns the user to DashboardForm and closes BarangayForm.

d. AlertForm.vb

```
Public Class AlertForm
```

- Starts the AlertForm class.
- This form manages:
 - Flood alerts
 - Alert issuing
 - Alert clearing
 - Alert monitoring

```
Private _alertSvc As New AlertService()
```

- Creates an instance of **AlertService**.
- This service handles:



- Issuing alerts
- Loading alerts
- Clearing alerts
- Deleting alerts

```
Private _floodSvc As New FloodStatusService()
```

- Creates an instance of **FloodStatusService**.
- Used to:
 - Check flooded barangays
 - Load barangay data

```
Public Property IsAdminMode As Boolean = False
```

- Stores whether the current user is an Admin.
- **True** → Admin controls enabled
- **False** → User view only

```
Me.StartPosition = FormStartPosition.CenterScreen
```

- Opens the form at the center of the screen.

```
LoadBarangays()
```

- Loads all barangays into the dropdown list.

```
SetupLevelComboBox()
```

- Loads the available alert levels into the alert level dropdown.
- Levels:
 - LOW
 - MODERATE
 - CRITICAL

```
RefreshGrid()
```

- Loads and refreshes the alerts DataGridView.



UpdateFloodStatus()

- Checks the number of flooded areas and updates the system status label.

```
PnlStatus.Visible = IsAdminMode  
btnClearAlerts.Visible = IsAdminMode  
btnDeleteAlert.Visible = IsAdminMode
```

- Shows Admin-only controls when the user is an Admin.
- Controls include:
 - Clear alerts
 - Delete alerts
 - Status management

```
cmbBarangay.Visible = IsAdminMode  
cmbLevel.Visible = IsAdminMode  
txtAlertMessage.Visible = IsAdminMode  
btnIssue.Visible = IsAdminMode
```

- Only Admins can:
 - Select barangays
 - Select alert levels
 - Write alert messages
 - Issue alerts

```
Dim dt As DataTable = _floodSvc.GetAll()
```

- Retrieves all barangays from **FloodStatusService**.

```
cmbBarangay.DataSource = dt
```

- Binds the barangay data to the ComboBox.

```
cmbBarangay.DisplayMember = "BarangayName"
```

- Displays the BarangayName column inside the dropdown.



```
cmbBarangay.ValueMember = "BarangayID"
```

- Stores the BarangayID as the selected value internally.

```
cmbLevel.Items.Clear()
```

- Clears existing items from the alert level ComboBox.

```
cmbLevel.Items.AddRange(New String() {"LOW", "MODERATE", "CRITICAL"})
```

- Adds the alert levels into the dropdown.

```
cmbLevel.SelectedIndex = 0
```

- Automatically selects the first item (**LOW**) by default.

```
Dim dt As DataTable = _alertSvc.GetActiveAlerts()
```

- Retrieves all active alerts from **AlertService**.

```
dgvAlerts.DataSource = Nothing  
dgvAlerts.Rows.Clear()  
dgvAlerts.Columns.Clear()
```

- Clears the existing DataGridView data before loading fresh records.

```
dgvAlerts.DataSource = dt
```

- Displays the retrieved alerts inside the DataGridView.

```
If dgvAlerts.Columns.Count > 0 Then
```

- Checks whether the DataGridView contains columns before formatting them.



```
If dgvAlerts.Columns.Contains("AlertID") Then
```

- Checks if the AlertID column exists.

```
dgvAlerts.Columns("AlertID").Visible = False
```

- Hides the AlertID column from users.
- The ID is used internally only.

```
dgvAlerts.AutoSizeColumnsMode = DataGridViewAutoSizeColumnsMode.Fill
```

- Automatically resizes all columns to fit the DataGridView width.

```
Console.WriteLine("Database returned no alerts.")
```

- Prints a message in the console if no alerts are returned.
- Used for debugging purposes.

```
Catch ex As Exception  
    MessageBox.Show("Grid Refresh Error: " & ex.Message)
```

- Handles errors while refreshing the grid.
- Displays the error message.

```
Dim dt As DataTable = _floodSvc.GetAll()
```

- Retrieves all barangays and their flood statuses.

```
Dim floodedCount As Integer = dt.Select("IsFlooded = True OR IsFlooded = 1").Length
```

- Counts the number of flooded barangays.
- Algorithm:
 - Filter rows where **IsFlooded = True**
 - Count matching rows

```
If floodedCount > 0 Then
```

- Checks whether at least one barangay is flooded.



```
lblStatusSummary.Text = $"⚠ STATUS: CRITICAL ({floodedCount} Areas Flooded)"
```

- Displays a critical warning with the number of flooded areas.

```
lblStatusSummary.ForeColor = Color.Red
```

- Changes the status label color to red for danger indication.

```
lblStatusSummary.Text = "✅ STATUS: ALL AREAS SAFE"
```

- Displays a safe status message when no flooded areas exist.

```
lblStatusSummary.ForeColor = Color.DarkGreen
```

- Changes the status label color to green for safety indication.

```
RefreshGrid()
```

- Reloads the latest alerts into the DataGridView.

```
Dim barangayId As Integer = Convert.ToInt32(cmbBarangay.SelectedValue)
```

- Gets the selected barangay ID from the ComboBox.

```
Dim message As String = txtAlertMessage.Text
```

- Retrieves the alert message entered by the Admin.

```
Dim level As String = cmbLevel.SelectedItem.ToString()
```

- Gets the selected alert level.

```
If _alertSvc.IssueAlert(barangayId, message, level) Then
```

- Calls the **IssueAlert()** method from **AlertService**.



- Business Logic:
 - Validate alert
 - Save alert to database
 - Return success/failure

```
MessageBox.Show("Alert Issued Successfully!", "FloodBuster")
```

- Displays a success message after creating an alert.

```
txtAlertMessage.Clear()
```

- Clears the alert message textbox after issuing the alert.

```
RefreshGrid()
```

- Refreshes the alerts list to show the newly created alert.

```
Dim confirm As DialogResult = MessageBox.Show(  
    "Are you sure you want to clear all active alerts?",  
    "Confirm Clear", MessageBoxButtons.YesNo, MessageBoxIcon.Warning)
```

- Displays a confirmation dialog before clearing all alerts.
- Prevents accidental clearing.

```
If confirm = DialogResult.Yes Then
```

- Runs the clear operation only if the user confirms.

```
If _alertSvc.ClearAll() Then
```

- Calls the **ClearAll()** method from **AlertService**.
- Marks all alerts as cleared.

```
MessageBox.Show("All alerts have been cleared.")
```



- Displays a success message after clearing alerts.

```
If dgvAlerts.CurrentRow Is Nothing Then
```

- Checks if an alert row is selected.

```
MessageBox.Show("Please select an alert to delete.", "No Selection",  
    MessageBoxButtons.OK, MessageBoxIcon.Warning)
```

- Warns the user when no alert is selected.

```
Exit Sub
```

- Stops the deletion process immediately.

```
Dim alertId As Integer = Convert.ToInt32(dgvAlerts.CurrentRow.Cells("AlertID").Value)
```

- Retrieves the selected alert ID.

```
Dim barangayName As String = dgvAlerts.CurrentRow.Cells("BarangayName").Value.ToString()
```

- Gets the barangay name from the selected alert row.

```
If _alertSvc.DeleteAlert(alertId) Then
```

- Deletes the selected alert using **AlertService**.

```
MessageBox.Show("Alert deleted successfully.", "FloodBuster",  
    MessageBoxButtons.OK, MessageBoxIcon.Information)
```

- Displays a success message after deleting the alert.

```
DashboardForm.Show()  
Me.Close()
```

- Returns the user to DashboardForm and closes AlertForm.



e. EvacuationForm.vb

```
Public Class EvacuationForm
```

- This defines the EvacuationForm class.
- This form is responsible for:
 - Selecting a barangay
 - Running evacuation recommendation logic
 - Displaying safest evacuation center results
- It is part of the **Presentation Layer (UI only)**.

```
Private _svc As New EvacuationService()
```

- Creates an instance of **EvacuationService** (Business Logic Layer).
- This service handles:
 - Building the graph of barangays
 - Running Dijkstra's algorithm
 - Computing evacuation recommendations
- So the form does NOT do calculations itself – it only calls the service.

```
Private Sub EvacuationForm_Load(sender As Object, e As EventArgs) Handles MyBase.Load  
    PopulateBarangayDropdown()  
End Sub
```

- Runs automatically when the form opens.
- Its only job:
 - Fill the dropdown with barangays so the user can select one

```
Private Sub PopulateBarangayDropdown()
```

- This method loads all barangays from the service into the ComboBox.

```
Dim dt As DataTable = _svc.GetAllBarangays()
```




- Calls the service layer to retrieve all barangays from the database.
- Returns a DataTable containing:
 - BarangayID
 - BarangayName

```
If dt.Rows.Count > 0 Then
```

- Checks if there is data available before binding it to the dropdown.
- Prevents errors when database is empty.

```
cboBarangay.DataSource = dt
```

- Binds the DataTable to the ComboBox.
- This allows the dropdown to display database records automatically.

```
cboBarangay.DisplayMember = "BarangayName"
```

- Sets what the user sees in the dropdown:
- Barangay names (e.g., "San Jose")

```
cboBarangay.ValueMember = "BarangayID"
```

- Stores the hidden value behind each item:
 - BarangayID (used internally for logic and database operations)

```
Private Sub btnGetRecommendation_Click(sender As Object, e As EventArgs) Handles btnGetRecommendation.Click
```

- Triggered when the user clicks "Get Recommendation".
- This is where evacuation routing begins.

```
Dim selectedLoc As String = cboBarangay.Text
```

- Gets the selected barangay name from the dropdown.



- This becomes the starting point for route calculation.

```
Dim dt As DataTable = _svc.GetRecommendations(selectedLoc)
```

- Calls the Business Logic Layer (EvacuationService).
- What it does internally:
 - Builds graph of barangays
 - Runs shortest path algorithm (Dijkstra)
 - Finds best evacuation centers
 - Returns ranked results

```
dgvResults.DataSource = dt
```

- Displays the recommendation results in the DataGridView.
- Each row represents an evacuation center option.

```
If dt.Rows.Count > 0 Then
```

- Checks if recommendations exist.
 - If yes → show best result
 - If no → show error message

```
lblStatus.Text = "Recommended: " & dt.Rows(0)("Evacuation Center").ToString()
```

- Displays the **top (best) evacuation center**.
 - Logic:
 - Row 0 = highest priority result
 - That is the recommended center

```
dgvResults.Rows(0).DefaultCellStyle.BackColor = Color.LightGreen
```

- Highlights the best recommendation in green.
- Purpose:
 - Makes it visually clear which center is best



```
Else  
    lblStatus.Text = "No path found for this location."  
End If
```

- If no routes are found:
 - Shows a message to the user
 - Means evacuation path is blocked or unavailable

```
Private Sub btnBack_Click(sender As Object, e As EventArgs) Handles btnBack.Click
```

- Triggered when user clicks Back.

```
DashboardForm.Show()  
Me.Close()
```

- Returns user to DashboardForm
- Closes EvacuationForm to free resources

f. ReportForm.vb

```
Imports CrystalDecisions.CrystalReports.Engine  
Imports CrystalDecisions.Shared  
Imports System.IO
```

- These are **library imports** used for Crystal Reports.
- **CrystalDecisions.CrystalReports.Engine**
 - Provides the ReportDocument class (core of Crystal Reports)
- **CrystalDecisions.Shared**
 - Used for database login, exporting, and report settings
- **System.IO**
 - Used for file handling (especially exporting PDFs)

```
Dim rpt As New FloodReport()
```

- Creates an instance of the Crystal Report file.
- **FloodReport()** is your **.rpt** design file



- This contains:
 - Tables
 - Layout
 - SQL queries
 - Fields for the report
- This object is the **main engine of the report system**.

```
Private Sub ReportForm_Load(sender As Object, e As EventArgs) Handles MyBase.Load  
    ConfigureReportConnection()  
End Sub
```

- Runs automatically when the ReportForm opens.
- It immediately calls:
 - ConfigureReportConnection()
- Meaning:
 - “Set up database connection and display report right away”

```
Private Sub ConfigureReportConnection()
```

- This is the core function of the report system.
- It handles:
 - Database connection
 - Data refresh
 - Binding report to viewer

Try

- Starts error handling.
- If something fails (like DB connection issues), it will be caught safely.

```
Dim connectionInfo As New ConnectionInfo()
```

- Creates a connection configuration object for Crystal Reports.

```
connectionInfo.ServerName = "MAVI\SQLEXPRESS"
```

- Specifies the SQL Server instance name.



- This tells Crystal Reports where the database server is located.

```
connectionInfo.DatabaseName = "FloodBusterDB"
```

- Defines the database name used by the report.

```
connectionInfo.IntegratedSecurity = True
```

- Uses Windows Authentication instead of username/password.
- Meaning:
 - Uses current Windows login credentials

```
For Each table As Table In rpt.Database.Tables
```

- Loops through every table inside the report.
- Why?, Because Crystal Reports may contain multiple tables, and all must be linked properly.

```
Dim tableLogOnInfo As TableLogOnInfo = table.LogOnInfo
```

- Gets the login configuration for each table.

```
tableLogOnInfo.ConnectionInfo = connectionInfo
```

- Assigns the correct database connection settings to the table.

```
table.ApplyLogOnInfo(tableLogOnInfo)
```

- Applies the connection settings.
- This ensures every table in the report can access the database.

```
rpt.Refresh()
```

- Refreshes the report data.
- Ensures:
 - Latest alerts



- Latest flood status
- Updated database records

```
rpt.RecordSelectionFormula = ""
```

- Removes filters from the report.
- Meaning:
 - Show ALL data (no restrictions)

```
CrystalReportViewer1.ReportSource = rpt
```

- Attaches the report to the viewer control.
- This is what displays the report on screen.

```
CrystalReportViewer1.RefreshReport()
```

- Refreshes the viewer so the report is rendered properly.

```
Catch ex As Exception
```

- Catches any errors (database, file, or report issues).

```
MessageBox.Show("Database Connection Error: " & ex.Message, "Error", MessageBoxButtons.OK, MessageBoxIcon.Error)
```

- Displays a user-friendly error message if report loading fails.

```
Private Sub btnExportPDF_Click(sender As Object, e As EventArgs) Handles btnExportPDF.Click
```

- Triggered when user clicks “Export PDF”.

```
Dim saveDialog As New SaveFileDialog()
```

- Opens a file save dialog so the user chooses where to store the PDF.

```
saveDialog.Filter = "PDF Files (*.pdf)|*.pdf"
```



IT ELECTIVE 1
School of Arts, Science & Technology
National Teachers College

- Restricts file type to PDF only.

```
saveDialog.FileName = "Official_Flood_Report_" & DateTime.Now.ToString("yyyyMMdd")
```

- Auto-generates a filename based on the current date.
- Example:
 - Official_Flood_Report_20260518.pdf

```
If saveDialog.ShowDialog() = DialogResult.OK Then
```

- Checks if user selected a valid save location.

```
rpt.ExportToDisk(ExportFormatType.PortableDocFormat, saveDialog.FileName)
```

- Exports the Crystal Report to a PDF file.

```
MessageBox.Show("Official PDF exported successfully!", "Export Complete")
```

- Confirms successful export to the user.

```
Catch ex As Exception
```

- Handles errors during export.

```
MessageBox.Show("Export failed: " & ex.Message, "Export Error")
```

- Shows error message if PDF export fails.

```
rpt.Close()  
rpt.Dispose()
```

- Free memory used by the report.
- Prevents:
 - Memory leaks
 - Locked report files

```
DashboardForm.Show()  
Me.Close()
```



- Returns the user to Dashboard and closes ReportForm.

BUSINESS LOGIC LAYER

a. AlertService

```
Public Class AlertService
    Private _alertRepo As New AlertRepository()

    ' Issue a new alert - called manually from AlertForm OR automatically from FloodStatusService
    2 references
    Public Function IssueAlert(barangayId As Integer, message As String, level As String) As Boolean
        If String.IsNullOrEmpty(message) Then
            MessageBox.Show("Alert message cannot be empty.", "Validation Error",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            Return False
        End If
        Return _alertRepo.Add(barangayId, message, level)
    End Function

    ' Get all active (uncleared) alerts for display
    1 reference
    Public Function GetActiveAlerts() As DataTable
        Return _alertRepo.GetActive()
    End Function

    ' Clear all alerts - Admin only (enforced in the UI)
    1 reference
    Public Function ClearAll() As Boolean
        Return _alertRepo.ClearAll()
    End Function

    ' Delete a specific alert by ID - Admin only (enforced in the UI)
    1 reference
    Public Function DeleteAlert(alertId As Integer) As Boolean
        Return _alertRepo.DeleteById(alertId)
    End Function
End Class
```

- IssueAlert(barangayId, message, level): Dispatches a new emergency alert (e.g., Warning, Critical). It ensures the message text isn't blank—showing a popup warning if it is—before saving it to the database via the repository.
- GetActiveAlerts(): Pulls a list of all current, unresolved alerts as a DataTable to show on active dashboards.
- ClearAll(): Wipes out or resolves all existing alerts simultaneously. (The comments note that the system relies on the user interface to ensure only an Administrator can click this).
- DeleteAlert(alertId): Permanently removes a single, specific alert from the database using its unique identifier.



b. EvacuationService

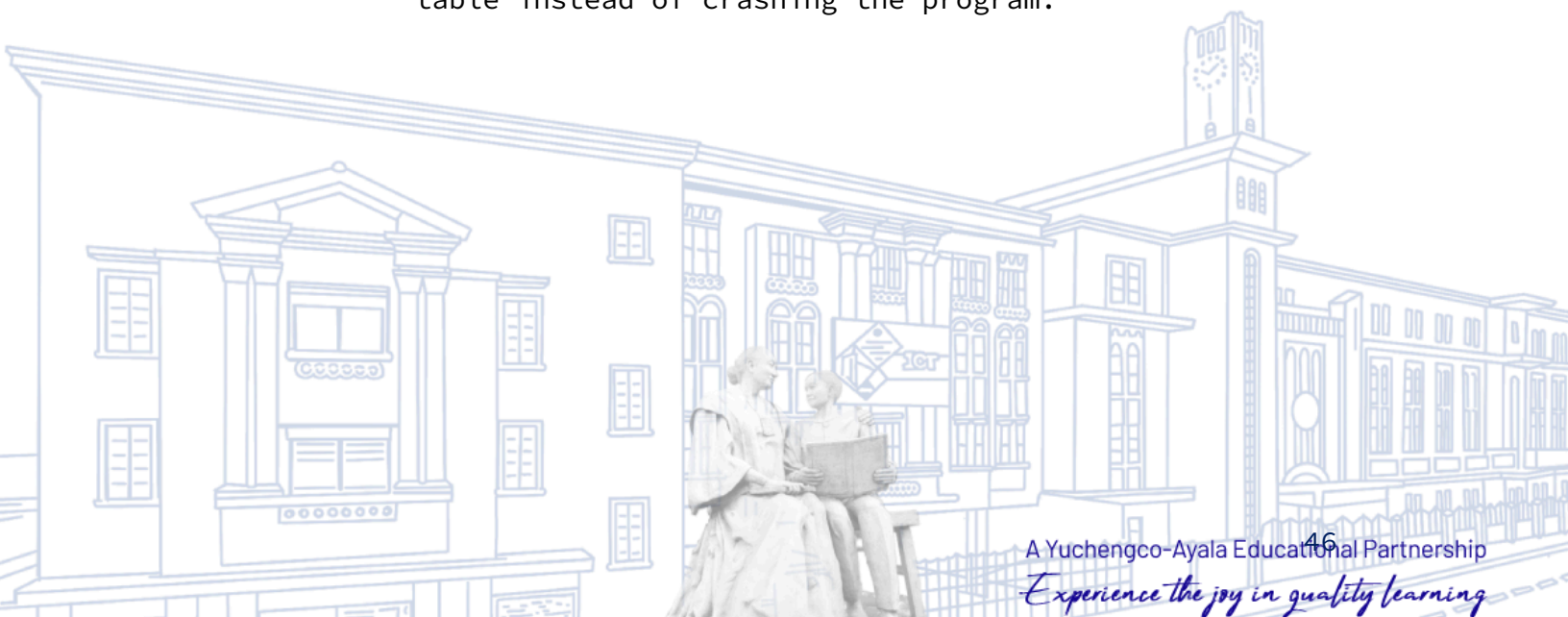
```
1 reference
Public Class EvacuationService

    Private _evacRepo As New EvacuationRepository()

    ' Get all barangays for the dropdown
    1 reference
    Public Function GetAllBarangays() As DataTable
        Return _evacRepo.GetAllBarangays()
    End Function

    ' Get evacuation path recommendations for a selected barangay
    1 reference
    Public Function GetRecommendations(barangayName As String) As DataTable
        If String.IsNullOrEmpty(barangayName) Then
            MessageBox.Show("Please select a barangay.", "Validation Error",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            Return New DataTable()
        End If
        Return _evacRepo.GetPathRecommendations(barangayName)
    End Function
End Class
```

- GetAllBarangays(): Fetches a list of all community areas from the database, typically used to populate selection dropdown menus (ComboBoxes) in the user interface.
- GetRecommendations(barangayName): Retrieves customized emergency evacuation routes and shelter recommendations based on a specific community's name.
 - Validation Rule: It checks if the barangayName parameter is empty. If no selection is made, it throws a warning popup and safely returns a blank, empty data table instead of crashing the program.





c. FloodStatusService

```
3 references
Public Class FloodStatusService

    Private _brgyRepo As New BarangayRepository()
    Private _alertService As New AlertService()

    ' Get all barangays (used by BarangayForm grid)
    3 references
    Public Function GetAll() As DataTable
        Return _brgyRepo.GetAll()
    End Function

    ' Add a new barangay - validates name is not empty
    1 reference
    Public Function AddBarangay(name As String) As Boolean
        If String.IsNullOrEmpty(name) Then
            MessageBox.Show("Barangay name cannot be empty.", "Validation Error",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            Return False
        End If
        Return _brgyRepo.Add(name)
    End Function

    ' Mark a barangay as flooded AND auto-generate a CRITICAL alert
    4 references

    ' Reset all barangays to Safe
    1 reference
    Public Function ResetAll() As Boolean
        Return _brgyRepo.ResetAll()
    End Function

    ' Delete a barangay
    1 reference
    Public Function DeleteBarangay(barangayId As Integer) As Boolean
        Return _brgyRepo.Delete(barangayId)
    End Function

End Class
```

- GetAll(): Retrieves a list of all barangays as a DataTable to display in a user interface grid.
- AddBarangay(name): Checks if the provided name is empty. If it is blank, it triggers a warning popup; if valid, it adds the new barangay to the system.
- MarkFlooded(barangayId, barangayName): Updates a specific location's status to "flooded" in the database. If successful, it automatically constructs and fires off a CRITICAL evacuation alert message through the alert service.
- ResetAll(): Resets all barangays back to a "Safe" status.
- DeleteBarangay(barangayId): Permanently removes a specific barangay from the system using its unique ID.



d. UserService

```
1 reference
Public Class UserService

    Private _userRepo As New UserRepository()

    ' Validate login - returns the role string ("Admin", "StandardUser") or "" if failed
    1 reference
    Public Function Login(username As String, password As String) As String
        1 reference
        If String.IsNullOrEmpty(username) OrElse String.IsNullOrEmpty(password) Then
            MessageBox.Show("Please enter both a username and password.", "Validation",
                MessageBoxButtons.OK, MessageBoxIcon.Warning)
            Return ""
        End If
        Return _userRepo.ValidateLogin(username, password)
    End Function

End Class
```

- Purpose: It acts as the business logic layer for authenticating users during login.
 - Login Function: Takes a username and password, verifies that neither field is empty, and displays a warning popup if they are.
 - Database Delegation: If the inputs are valid, it passes them to the database layer (_userRepo.ValidateLogin) to check the credentials and return the user's account role (like "Admin" or "StandardUser").

DATABASE ACCESS LAYER

a. DataConnection.vb

```
Imports System.Data.SqlClient

16 references
Public Class DatabaseConnection

    Private Const CONNECTION_STRING As String = "Server=MAVI\SQLEXPRESS;Database=FloodBusterDB;Integrated Security=True;TrustServerCertificate=1

    16 references
    Public Shared Function GetConnection() As SqlConnection
        Return New SqlConnection(CONNECTION_STRING)
    End Function

End Class
```

- This class serves as a centralized manager to automatically configure and hand out database connections to the rest of the application.



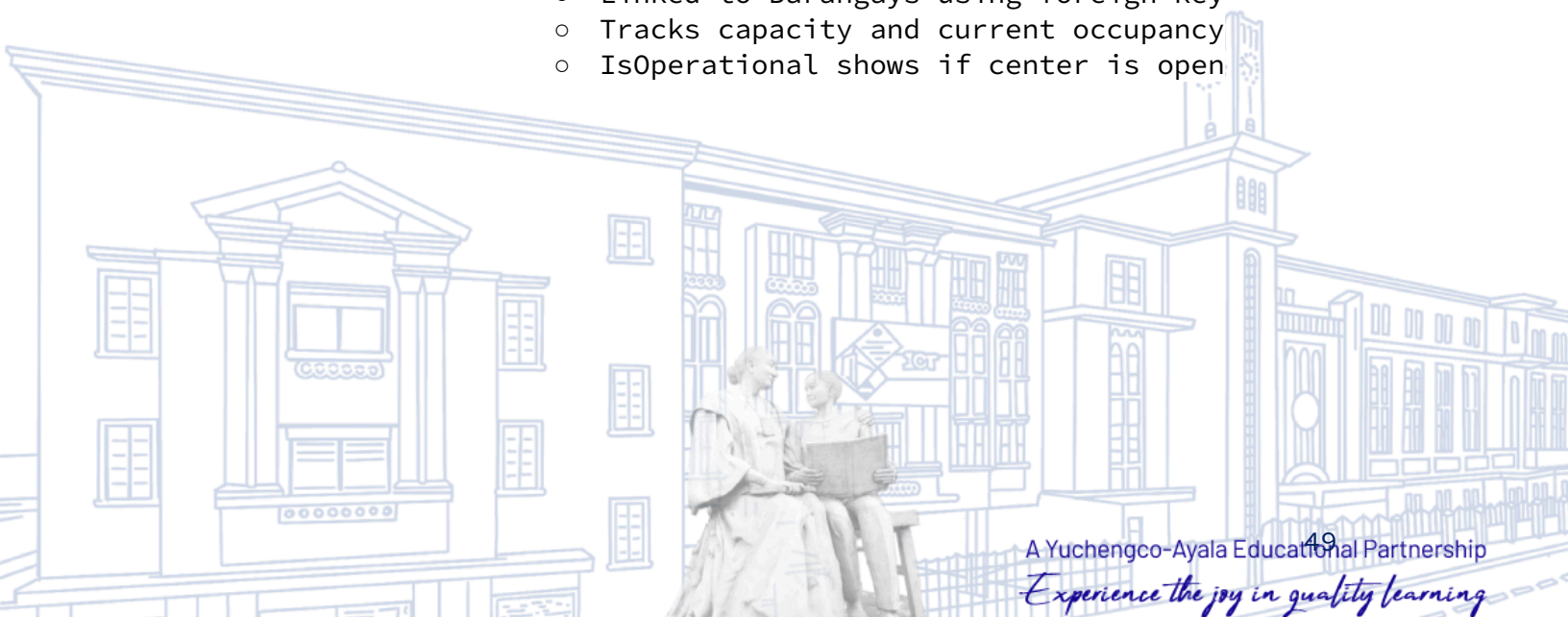
b. Tables

```
CREATE TABLE Barangays (  
    BarangayID INT PRIMARY KEY IDENTITY(1,1),  
    BarangayName NVARCHAR(100) NOT NULL UNIQUE,  
    IsFlooded BIT NOT NULL DEFAULT 0,  
    LastUpdated DATETIME NOT NULL DEFAULT GETDATE()  
);
```

- This table stores all barangays in the system.
 - BarangayID → unique ID for each barangay
 - BarangayName → name of the barangay
 - IsFlooded → shows if the area is flooded (0 = safe, 1 = flooded)
 - LastUpdated → automatically records the last update time

```
CREATE TABLE EvacuationCenters (  
    CenterID INT PRIMARY KEY IDENTITY(1,1),  
    CenterName NVARCHAR(150) NOT NULL,  
    BarangayID INT NOT NULL,  
    MaxCapacity INT NOT NULL,  
    CurrentOccupancy INT NOT NULL DEFAULT 0,  
    IsOperational BIT NOT NULL DEFAULT 1,  
    CONSTRAINT FK_Evacuation_Barangay FOREIGN KEY (BarangayID)  
    REFERENCES Barangays(BarangayID)  
);
```

- Stores evacuation centers for each barangay.
 - Linked to Barangays using foreign key
 - Tracks capacity and current occupancy
 - IsOperational shows if center is open





```
CREATE TABLE FloodAlerts (  
    AlertID INT PRIMARY KEY IDENTITY(1,1),  
    BarangayID INT NOT NULL,  
    AlertMessage NVARCHAR(500) NOT NULL,  
    AlertLevel NVARCHAR(20) NOT NULL,  
    DateIssued DATETIME NOT NULL DEFAULT GETDATE(),  
    IsCleared BIT NOT NULL DEFAULT 0,  
    CONSTRAINT FK_Alerts_Barangay FOREIGN KEY (BarangayID)  
    REFERENCES Barangays(BarangayID)  
);
```

- Stores flood alert messages.
 - Contains alert message and severity level
 - Tracks if alert is already cleared
 - Connected to Barangays table

```
CREATE TABLE Users (  
    UserID INT PRIMARY KEY IDENTITY(1,1),  
    Username NVARCHAR(50) NOT NULL UNIQUE,  
    PasswordHash NVARCHAR(256) NOT NULL,  
    Role NVARCHAR(20) NOT NULL,  
    DateCreated DATETIME NOT NULL DEFAULT GETDATE()  
);
```

- Stores system users for login. \
 - Username must be unique
 - PasswordHash stores encrypted password
 - Role determines Admin or User access

```
CREATE TABLE BarangayConnections (  
    ConnectionID INT PRIMARY KEY IDENTITY(1,1),  
    FromBarangayID INT NOT NULL,  
    ToBarangayID INT NOT NULL,  
    DistanceMeters INT NOT NULL,  
    CONSTRAINT FK_From_Barangay FOREIGN KEY (FromBarangayID)  
    REFERENCES Barangays(BarangayID),  
    CONSTRAINT FK_To_Barangay FOREIGN KEY (ToBarangayID)  
    REFERENCES Barangays(BarangayID)  
);
```

- Stores road connections between barangays.
 - Used for evacuation path calculations



- Contains distance between two barangays

c. EvacuationRepository

GetAllBarangays() (READ):

- Pulls a list of all community IDs and names from the Barangays table, sorted alphabetically.
- It is intentionally built to populate user interface dropdowns (ComboBox controls), ensuring both the display text (Name) and behind-the-scenes value (ID) are loaded correctly.

GetPathRecommendations(userBarangay) (READ - Dynamic Query):

- This function runs a multi-table database JOIN connecting evacuation centers, community lists, and mapping connections.
- It accepts the user's specific location (@UserLoc) to securely filter and calculate available evacuation targets.
- Dynamic Column Output: It tracks and pulls the Evacuation Center's name, its physical location, the path distance in meters, a safe road route description, and computes real-time vacancy space by running a math calculation directly inside the query: (MaxCapacity - CurrentOccupancy).

d. BarangayRepository

- GetAll() & GetCriticalAreas() (READ):
 - GetAll pulls a list of all locations sorted by name to display on a management grid.
 - GetCriticalAreas specifically targets and filters only the areas currently marked as flooded (IsFlooded = 1).
- Add(name) (CREATE): Inserts a new community area into the system, defaulting its initial status flag to safe (IsFlooded = 0).
- MarkFlooded(id) & ResetAll() (UPDATE):



- MarkFlooded flags a specific location as active by switching IsFlooded = 1 and recording the server timestamp (GETDATE()).
 - ResetAll batch updates every single community back to a safe state (IsFlooded = 0) simultaneously.
- Delete(id) (DELETE): Performs a cascading database cleanup. Because of foreign key constraints, it executes a two-step process: it first drops any related entries in the FloodAlerts table before safely removing the target community row itself.

e. AlertRepository

- GetActive() (READ): Runs a SELECT query with an INNER JOIN to fetch all active flood alerts (IsCleared = 0) along with their matching Barangay names. It loads the results into a DataTable using a SqlDataAdapter.
- Add (CREATE): Inserts a new alert record into the FloodAlerts table. It safely utilizes parameterized inputs (@bid, @msg, @lvl) to block SQL Injection attacks.
- ClearAll() (UPDATE): Switches the IsCleared status flag to 1 (true) for every single active alert in the system simultaneously.
- DeleteById (SOFT DELETE): Instead of wiping the data entirely via a hard DELETE query, this function performs a soft delete by changing the single target alert's IsCleared flag to 1. This keeps historical data intact while removing it from the active user view.

f. UserRepository

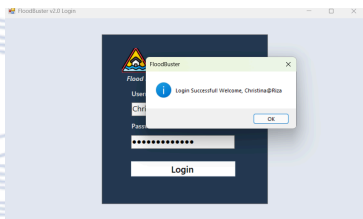
- ValidateLogin(username, password) (AUTHENTICATION):
 - Queries the database to look for a row where both the username and password hash match the inputs.
 - Uses cmd.ExecuteScalar() because it only needs to pull a single value (the user's Role string) rather than a whole table row.
 - If a match is found, it returns the role (e.g., "Admin" or "StandardUser"). If the credentials are wrong, it returns an empty string ("").



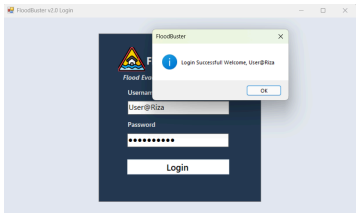
- GetAll() (READ):
 - Selects basic profile information (UserID, Username, Role, DateCreated) for all registered users, sorted alphabetically.
 - It leaves out sensitive data like password hashes, making the output perfect to securely bind to a management grid for system administrators.
- Add(username, passwordHash, role) (CREATE):
 - Inserts a brand new account record into the Users table.
 - The comments note that the database table features a check constraint requiring the role parameter to explicitly be either "Admin" or "StandardUser".
- Delete(userId) (DELETE):
 - Executes a hard delete (DELETE FROM) statement to permanently wipe out an account from the database table matching the provided system ID.

VI. TESTING AND RESULT

Scenario 1: Login - This section documents the test cases, expected behaviors, and actual user interface updates during system login verification.

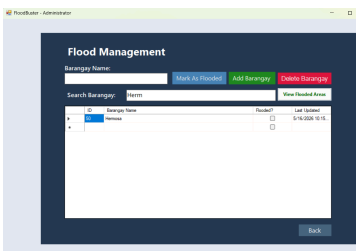
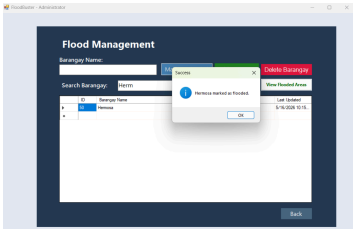
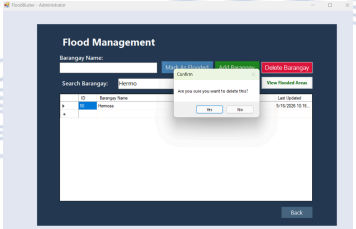
Actual Outputs	Expected Result	Steps conducted
Test Case 1: Administrative Credential Authentication Objective: Verify that an administrator login opens the dashboard with all functional controls visible.		
	• A confirmation message box pops up: "Login Successful! Welcome Admin" • LoginForm hides • DashboardForm	• Launch application. • Enter an authorized admin username into Username. • Enter the correct password into Password. • Click Login.



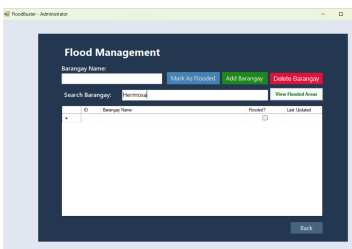
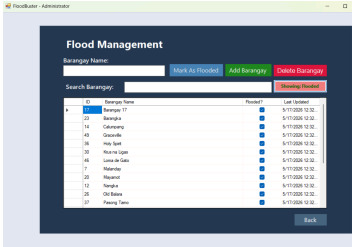
	opens with all operations (btnMarkFlooded , btnGenerateReport) visible.	
Test Case 2: Standard User Credential Authentication Objective: Verify that a regular user login grants access but restricts administrative buttons.		
	• A confirmation message box pops up: "Login Successful! Welcome User" • LoginForm hides • DashboardForm opens but hides admin keys (btnMarkFlooded, btnGenerateReport).	• Launch application. • Enter an authorized admin username into Username. • Enter the correct password into Password. • Click Login.

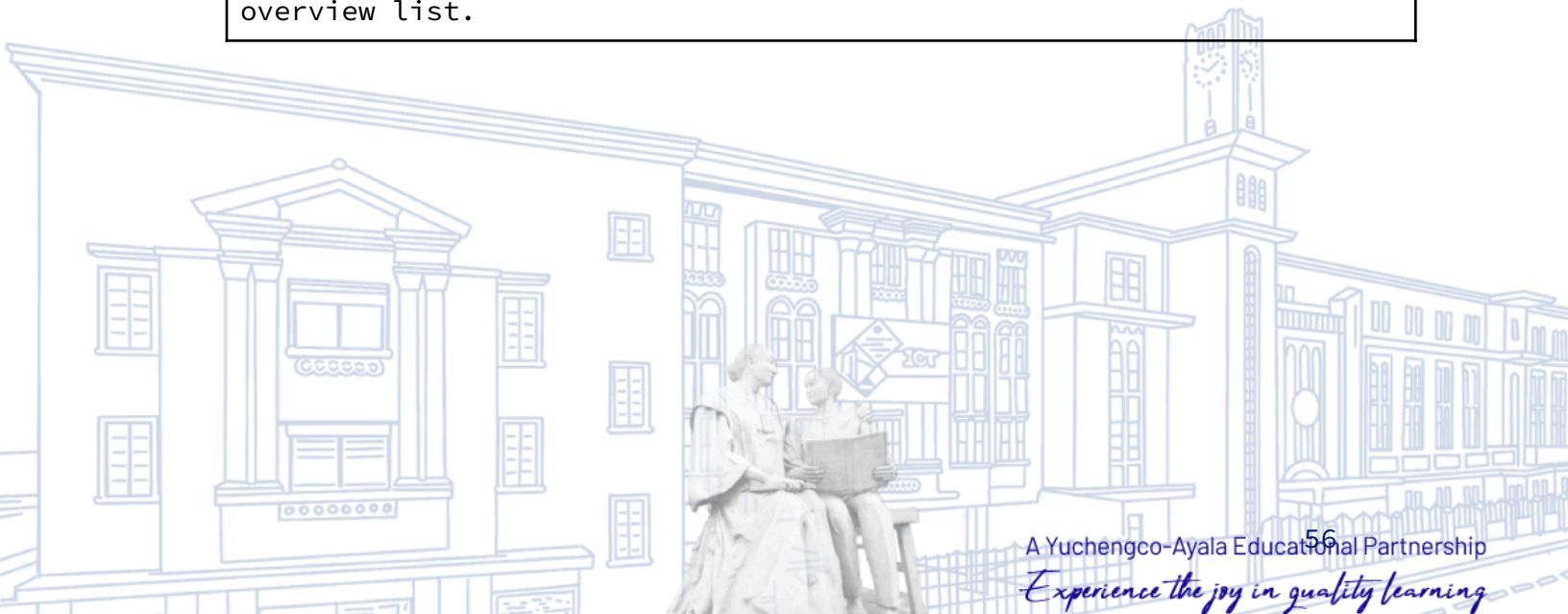
Scenario 2: (ADMIN) Mark as Flooded - This section documents the test cases, expected system behaviors, and actual database results when managing records with an administrator account.		
Actual Outputs	Expected Result	Steps conducted
Test Case 1: Marking Areas as Flooded Objective: Verify that selecting a location and clicking "Mark as flooded" immediately updates its status in the database and creates a system alert.		



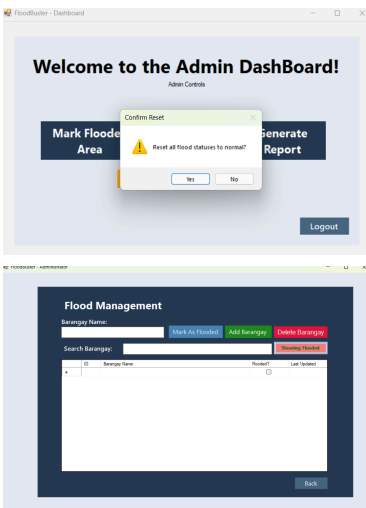
	• “Hermosa is marked as flooded.” • Message box pops out to confirm the barangay is marked as flooded	• Navigate to the Admin Dashboard. • Select in the data grid view or in the table directly • Highlight "Hermosa" inside the table. • Click the "Mark as flooded" button.
Test Case 2: Adding a New Barangay (Create) Objective: Verify that an administrator can successfully add a new barangay to the system using safe, parameterized queries.		
	• The new Barangay should appear after pressing “Add Barangay” • Refreshes data-bindings for dgvBarangays to cleanly display the newly appended row.	• Type a new location name into Barangay Name. • Click Add Barangay Button • Input a target search keyword into the search bar.
Test Case 3: Deleting a Barangay (Delete) Objective: Verify that deleting a barangay safely clears its linked alerts first to prevent database errors (foreign key constraint violations).		
	• Generates a warning dialogue reading: "Are you sure you want to delete this?"	• Select a target row inside the data grid or the table . • Click Delete Barangay Button • Click "OK" on the confirmation prompt



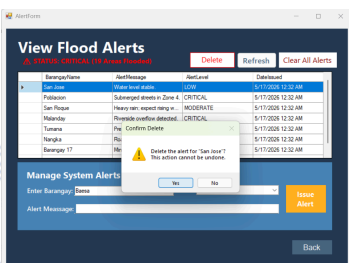
	• The grid updates; the entry vanishes completely from subsequent queries.	window.
Test Case 4: Dynamic State-Based Grid Filtering Objective: Verify that clicking the status filter button restricts the data grid view to exclusively display active flood mutation records.		
	• The grid contents refresh, hiding all normal records.	• Navigate to the Flood Management sub-form. • Locate the active database grid panel. • Left-click the "Showing: Flooded" status toggle button.
Test Case 5: Global System Status Reset (Bulk State Reversal) Objective: Verify that triggering the master reset button resets all location statuses to normal and completely clears the active flood overview list.		

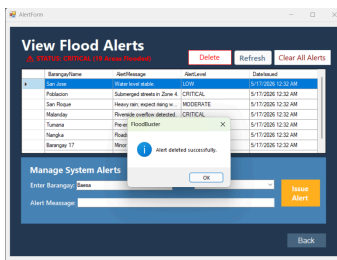




	• A confirmation message box pops up: <i>"Reset all flood statuses to normal?"</i> • The filtered "Showing: Flooded" view now shows an entirely empty log layout.	• Open the Admin Dashboard. • Click the yellow system reset button. • Click "Yes" on the confirmation prompt window. • Navigate to the Flood Management sub-form to review active records.
---	--	---

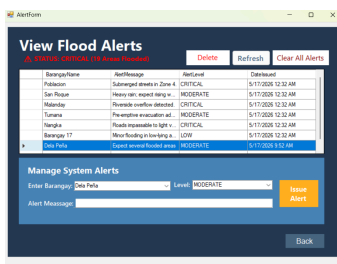
Scenario 3: View Flood Alerts - This section documents the test cases, expected system behaviors, and actual user interface updates when administrators issue or clear emergency alerts.

Actual Outputs	Expected Result	Steps conducted
Test Case 1: Individual Alert Deletion (Cascading Drop) Objective: Verify that selecting an alert entry and clicking "Delete" safely prompts for confirmation, executes a targeted database wipe, and removes the row from the data grid.		
	• A prompt displays asking: <i>"Delete the alert for 'San Jose'? This action cannot be undone."</i> • Displays a modal popup stating: <i>"Alert deleted successfully."</i>	• Open the View Flood Alerts form. • Select the record for "San Jose" from dgvAlerts. • Click the "Delete" button. • Click "Yes" on the confirmation prompt window.



Test Case 2: Manual Parameterized Alert Issuance

Objective: Verify that an administrator can manually broadcast a custom, state-bound warning alert for a selected barangay.

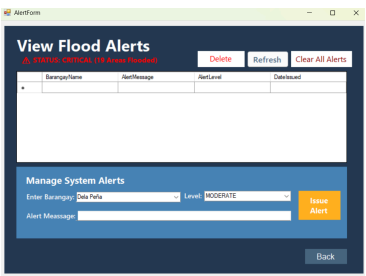


- The main view updates immediately, appending the new row to the bottom of the log list.
- Select **"Dela Peña"** from the drop-down combo box.
- Select **"MODERATE"** from the Level drop-down list.
- Input custom message details.
- Click the **"Issue Alert"** button.

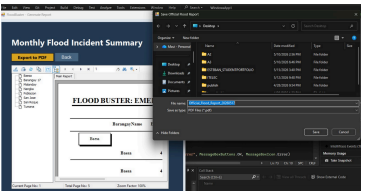
Test Case 3: Complete Alert Log Truncation

Objective: Verify that clicking the "Clear All Alerts" button prompts the admin for verification before wiping out the entire active alert table schema.



	• A warning box pops up: "Are you sure you want to clear all active alerts?" • Clears dgvAlerts completely, leaving it empty.	• Open the View Flood Alerts form. • Click the "Clear All Alerts" button. • Click "Yes" on the warning confirmation dialog box.
---	---	---

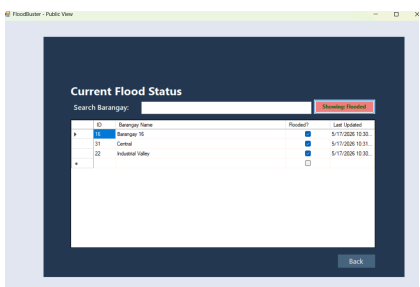
Scenario 4:Administrative Report Generation – This section documents the test case, system operations, and verified document generation results when an administrator compiles official summary reports.

Actual Outputs	Expected Result	Steps conducted
Test Case 1: Crystal Report Generation Objective: Verify that clicking the "Generate Report" button correctly compiles historical log data into the Crystal Report viewer component and exports it safely into a non-editable PDF document. 	• The system fetches live data streams and groups them cleanly by location name inside the Crystal Report viewer (crystalReport Viewer1) • Clicking the export trigger interrupts focus with a native system	• Click "Generate Report" from the Admin Dashboard. • Once the report window initializes, click the "Export to PDF" button. • Choose a destination folder (e.g., Desktop), verify the automatic filename template (Official_Flood_Report_20260517), and



	save dialog utility defaulting to Save as type: PDF Files (*.pdf).	click "Save" .
--	--	-----------------------

Scenario 5: Standard User Interface Validation - This section documents the test cases, system actions, and database retrieval behaviors when logging in with a standard user profile. All administrative actions (Create, Update, Delete) are blocked, making these interfaces strictly read-only.

Actual Outputs	Expected Result	Steps conducted
Test Case 1: Dashboard Role Isolation (Read-Only Access) Objective: Verify that standard user credentials restrict administrative buttons and display only client utility paths.		
	The dashboard hides administrative keys like "Mark Flooded Area", "Add Barangay", or the master reset toggle.	- Log into the application using a standard user account. - Click "View Flood Status" from the User Options dashboard. - Use the search bar to query a location name. - Toggle the status filter.

Test Case 3: Read-Only Active Alert Stream

Objective: Verify that the system pulls live broadcast notices from the database table cleanly while blocking data management controls.



Barangay/Name	AlertMessage	AlertLevel	DateIssued
Inda, Ipul	Please evacuate to your nea...	CRITICAL	5/17/2020 10:12 AM
Santa Lucia	Multiple roads are currently c...	CRITICAL	5/17/2020 10:13 AM
Tumana	Stay tuned for real-time com...	LOW	5/17/2020 10:18 AM
Barangay 16	Barangay 16 is FLOODED...	CRITICAL	5/17/2020 10:30 AM
Industrial Valley	Industrial Valley is FLOODI...	CRITICAL	5/17/2020 10:30 AM
Central	Central is FLOODED. Please...	CRITICAL	5/17/2020 10:31 AM

- Populates the alert log window smoothly.
- Management buttons like **"Delete"**, **"Clear All Alerts"**, and the issue fields are missing.

- Click **"View Alerts"** from the User Options dashboard.
- Click the **"Refresh"** button to look for updates.

Test Case 4: Evacuation Path Routing and Center Capacity Verification

Objective: Verify that selecting a source barangay executes a path-finding search to identify, rank, and recommend the nearest operational evacuation center with remaining occupancy.

Evacuation Center	Location	Distance	Safe Path	Available Slots
Tumana Disaster	Tumana	500	Malanday-Tumana	50

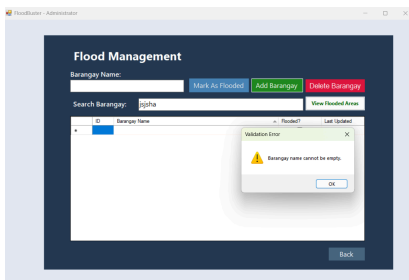
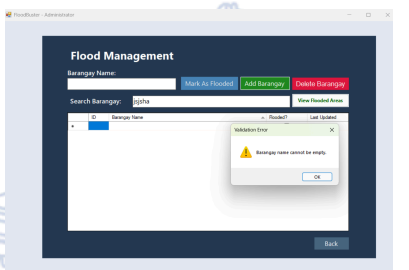
- Ranks optimal locations by shortest distance and populates **dgvResults** smoothly.
- **System Recommendation:** Highlights the top entry and sets **lblStatus** to: **"Recommended: Tumana"**

- Open the Evacuation Path Recommendation form.
- Select a barangay from the **cboBarangay** drop-down list.
- Click the **"Get Recommendation"** button.



	Disaster Center"	
--	---------------------	--

Scenario 6: Error and Exception Handling Validation - This section captures the test cases, structural execution steps, and system exception controls used to process incorrect inputs and invalid actions safely without causing runtime application crashes. read-only.

Test Case	Steps conducted	Expected Result	Actual output
1. Empty Barangay Insertion Attempt	1. Open the Admin Dashboard. 2. Leave the txtBarangayName text field completely blank. 3. Click the "Add Barangay" button.	The system halts the execution thread immediately before sending a command to the database and alerts the operator.	
2. Unselected Record Deletion Exception Check	1. Open the View Flood Alerts window (AlertForm.vb). 2. Do not highlight or click any active rows inside the	The frontend validation interceptor catches the null pointer context, cancels communication with the database engine, and	



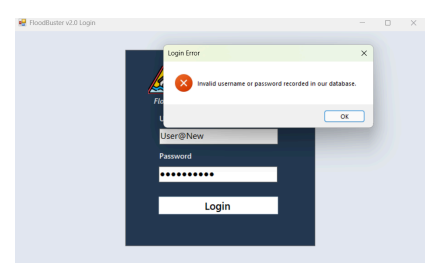
	alert data grid container (dgvAlerts). 3. Click the "Delete" button immediately.	instructs the administrator to make a valid selection.	
3. Empty Text Input Rejection	1. Navigate to the Manage System Alerts pane inside AlertForm.vb. 2. Select a valid barangay ("Santa Lucia") from the input drop-down menu. 3. Leave the "Alert Message" input field completely empty. 4. Click the "Issue Alert" action button.	The application catches the empty string context during the preprocessing validation phase and stops an incomplete record creation query from firing.	



**4.
Unregistered
Key Logon Test**

1. Launch the LoginForm.vb startup window.
2. Type an unregistered profile username, such as "User@New", into the input box.
3. Input an invalid password combination into the secure field.
4. Click the "Login" action button.

The authentication algorithm evaluates the keys using a safe parameter query, discovers that zero matching records exist, and prevents entry to the dashboard.



VII. CONCLUSION

The Flood Evacuation Path Planning System (FloodBuster V2.0) successfully delivers a high-quality, data-centric VB.NET Windows Forms Application that addresses critical disaster preparedness needs in the Philippines. Built using a robust N-Tier architecture consisting of Presentation, Business Logic, and Data Access layers, and integrated with a



persistent SQL Server database, the system effectively meets all of its structural and functional objectives.

The application incorporates secure, parameterized CRUD operations to eliminate SQL injection vulnerabilities alongside an isolated, repository-based architecture for enhanced project maintainability. Robust role-based access control guarantees that sensitive operations—such as marking areas as flooded, clearing alerts, and generating reports—are securely restricted to Administrators, while providing Standard Users with seamless, intuitive tools for route planning and public alert monitoring.

Furthermore, through the execution of intelligent evacuation path routing and the seamless implementation of filtered Crystal Reports, the system provides real-time, actionable insights for local government units and barangay officials. Ultimately, by introducing a structured, automated, and secure platform for managing flood alerts and routing evacuations, FloodBuster V2.0 directly and successfully aligns with the targets of UN Sustainable Development Goal 11 (Sustainable Cities and Communities) by working toward lowering the impact of water-related natural disasters on vulnerable communities



VIII. CONTRIBUTION

A table detailing exactly which module/form was implemented by which member.

Member	Core Project Roles & Technical Contributions
Caliguiran, Arjane	Business Logic Layer (BLL), Module Breakdown
Esteban, Maria Victoria	UI/UX Design, Crystal Report Implementation, Database Development, GitHub Management, Co-Compiler & Organizer
Lagrimas, Don Christian	Entity-Relationship Diagrams (ERDs), Database Development
Lamayo, Justine Kurt	System Architecture Diagram, Data Flow Diagrams (DFDs), Presentation Deck (PPT)
Riza, Christina Alexandra	UI/UX Design, Documentation Organizer & Compiler, Testing & Result